Simulateur IoT en C++17

Rapport de TP AP4A - Automne 2026

Étudiant : FABRIZIO Elio - Groupe TP4B (semaine B)

1. Présentation du projet

Ce projet consiste à simuler plusieurs capteurs installés dans un espace de travail. Ils mesurent la température, l'humidité, le bruit et l'état de la lumière. Leurs données sont envoyées à un serveur, qui les affiche, les enregistre et en tire quelques statistiques. Ce cadre permet de mettre en pratique les notions de programmation orientée objet vues en AP4A, avec le standard C++17.

Pour représenter le passage du temps, le programme utilise des cycles appelés « ticks ». À chaque cycle, l'ordonnanceur sollicite les capteurs, mais chacun ne prend une mesure que lorsque sa période est atteinte. Les échanges avec le serveur se font par appels de méthodes : aucun réseau n'est nécessaire. Un tick n'a pas de durée fixée en secondes ; il sert simplement à faire avancer la simulation.

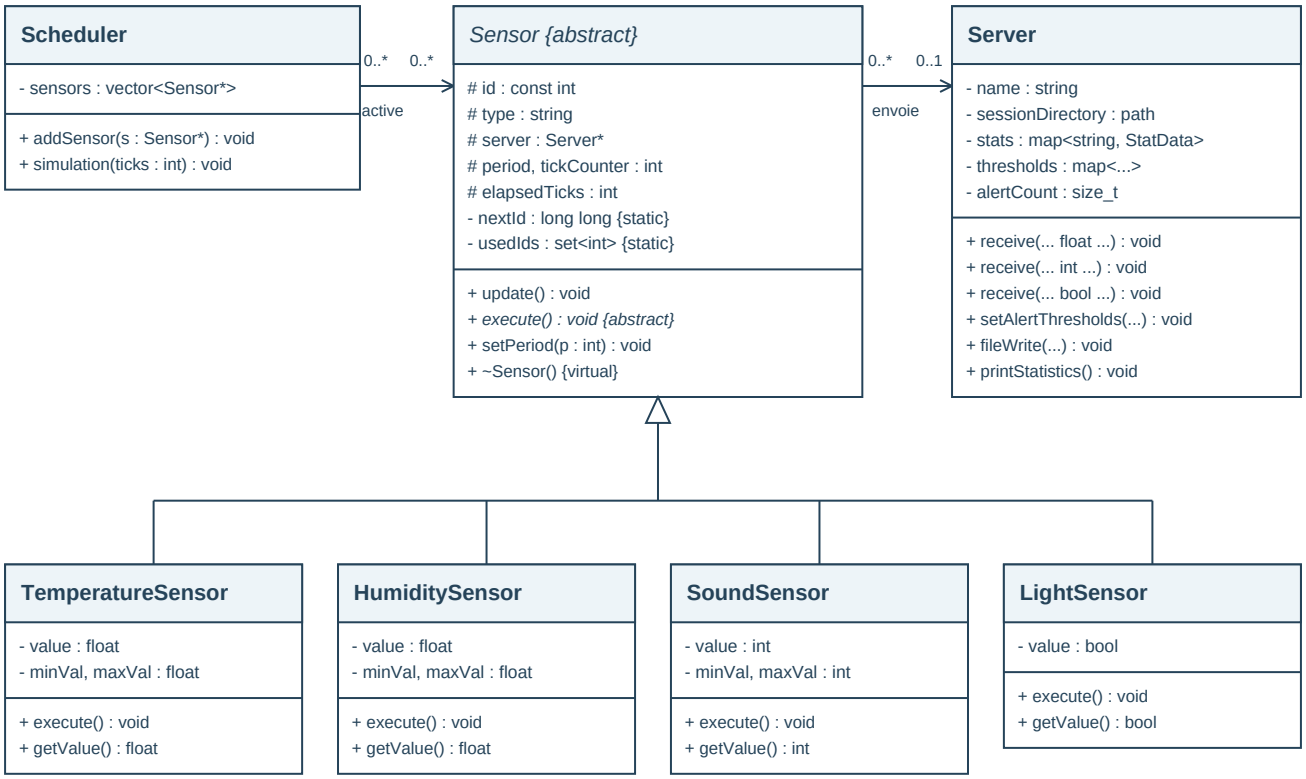
Les principales attentes du sujet

Attente du sujet	Réalisation
Serveur et ordonnanceur	Classes Server et Scheduler distinctes
Capteurs et valeurs hétérogènes	Sensor abstraite ; float, int et bool
Héritage et polymorphisme	Quatre spécialisations ; execute virtuelle pure
Périodes propres aux capteurs	Compteur interne et période configurable
Console et fichiers par type	Réception centralisée et CSV séparés
Opérateurs de flux	Console et inventaire fichier
Compilation portable	CMake, C++17, bibliothèque standard
Extensions d'analyse	Statistiques et alertes sur seuils

Organisation du code

Le dossier `include/` contient les déclarations des classes et `src/` leur implémentation. Le fichier `main.cpp` crée les objets et lance la démonstration. Les tests sont regroupés dans `tests/test_simulator.cpp`. Enfin, CMake permet de compiler l'ensemble et de lancer les vérifications avec CTest ; les commandes utiles sont rappelées dans le README.

2. Architecture et diagramme UML



Le rôle de chaque classe

- **Sensor** regroupe ce que les capteurs ont en commun : un identifiant, un lien vers le serveur et des compteurs. Sa méthode `update()` appelle `execute()` lorsqu'une mesure doit être prise.
- **Les quatre classes de capteurs** définissent chacune leur façon de produire une valeur, puis l'envoient au serveur avec `receive()`.
- **Scheduler** parcourt sa liste de capteurs et appelle `update()` sur chacun. Il n'a pas besoin de savoir s'il s'agit d'une température ou d'un niveau sonore. Un même objet ne peut pas être inscrit deux fois dans sa liste.
- **Server** s'occupe de l'affichage, des fichiers CSV, des statistiques et des alertes. Il reçoit aussi l'unité et le type de valeur, ce qui lui permet de traiter les mesures sans prévoir un affichage spécifique à chaque nom de capteur.

Cette répartition facilite l'ajout d'un capteur : il faut créer sa classe, l'ajouter à la compilation et l'instancier dans le programme. L'ordonnanceur peut alors le piloter sans modification. Le serveur peut aussi réutiliser ses calculs si les données envoyées restent numériques ou booléennes, avec une unité cohérente.

3. Les choix de programmation objet

Héritage, polymorphisme et surcharge

La classe `Sensor` est abstraite : sa méthode `execute() = 0` indique que chaque capteur doit définir sa propre façon de mesurer. Les classes dérivées le font avec le mot-clé `override`. Ainsi, lorsque `update()` appelle `execute()`, c'est bien la version du capteur concerné qui est utilisée. Le destructeur de Sensor est virtuel pour permettre une destruction correcte à travers un pointeur de base.

La méthode `receive()` du serveur est surchargée pour accepter des `float`, des `int` et des `bool`. Ces trois versions rejoignent ensuite le même traitement pour sauvegarder les données et calculer les statistiques. Les opérateurs << permettent, eux, d'afficher les objets avec une syntaxe simple. Le programme les utilise aussi bien dans la console que dans un fichier d'inventaire.

Forme de Coplien et identité

Les classes principales suivent la forme de Coplien : constructeur par défaut, constructeur de copie, opérateur d'affectation et destructeur. Le point qui demande le plus d'attention est le sens donné à la copie. Copier un capteur doit créer un objet distinct, alors que copier son pointeur vers le serveur doit conserver le lien avec le même serveur.

- La copie d'un capteur reçoit un nouvel identifiant, mais reprend sa valeur, ses compteurs et son lien vers le serveur.
- Une affectation conserve l'identifiant du capteur qui reçoit la copie. Affecter un objet à lui-même est également pris en charge.
- Un identifiant déjà utilisé ne peut pas être réattribué pendant l'exécution, même après la destruction du capteur.
- La copie d'un ordonnanceur garde des pointeurs vers les mêmes capteurs. Faire tourner cette copie fait donc avancer les objets d'origine.
- La copie d'un serveur reprend ses statistiques, ses seuils et son compteur d'alertes, tout en écrivant dans le même dossier. Pour démarrer une simulation séparée, il faut créer un nouveau serveur.

Gestion de la mémoire

Les objets sont créés directement dans `main`, sans allocation dynamique. Le serveur est créé en premier, puis les capteurs et enfin l'ordonnanceur. Comme ils sont détruits dans l'ordre inverse, le serveur reste disponible aussi longtemps que les capteurs en ont besoin.

Les pointeurs de Sensor et Scheduler sont dits « non propriétaires » : ils donnent accès à un objet, mais ne sont pas chargés de le détruire. Ce choix reste simple pour cette démonstration, à condition de respecter les durées de vie. Les attributs sont privés ou protégés, et l'identifiant ne change plus après la construction.

4. Déroulement de la simulation

Démonstration sur six ticks

Capteur	Donnée et plage	Période	Mesures
Température bureau 1	float, 12 à 26 degC	2 ticks	3
Température bureau 2	float, 26 à 28 degC	3 ticks	2
Humidité	float, 35 à 45 %	3 ticks	2
Son	int, 72 à 82 dB	1 tick	6
Lumière	bool, 0 ou 1	1 tick	6

Les deux capteurs de température produisent donc cinq mesures au total, regroupées dans le même bilan. Le scénario représente volontairement un bureau chaud, sec et bruyant pour montrer les alertes. Au départ, chaque valeur numérique est placée au milieu de sa plage, avec un arrondi vers zéro pour le son. Elle n'est enregistrée qu'une fois `execute()` appelée.

Pour éviter des sauts peu crédibles, les valeurs évoluent à partir de la mesure précédente. La variation est limitée à 0,5 degC, 2 points d'humidité ou 5 dB par mesure, sans sortir de la plage choisie. Cette marche aléatoire utilise `std::mt19937`, initialisé avec `std::random_device`. La lumière reste un tirage indépendant entre 0 et 1. Si les deux bornes d'une plage sont égales, la valeur reste fixe.

Comment sont comptés les ticks ?

Chaque capteur compte les appels à `update()` qu'il a reçus, y compris entre deux appels à `simulation()`. Dans la démonstration, tous commencent ensemble. Un capteur ajouté plus tard aurait toutefois son propre décompte : il n'y a pas encore d'horloge globale. Un appel direct à `execute()` peut aussi enregistrer une mesure au tick 0. Enfin, changer la période remet à zéro le compteur qui détermine la prochaine mesure.

Journaux et statistiques

À chaque nouveau serveur correspond un dossier dans `logs/`, ce qui évite de mélanger les résultats de plusieurs exécutions. Seules les copies d'un même serveur partagent ce dossier. Les mesures sont réparties dans un fichier CSV par type de capteur, avec le format suivant :

```
Tick;SensorID;Type;Valeur;Unite
2;1;Temperature;20;degC
```

Cet exemple montre le format d'une ligne. Pour ouvrir le fichier dans un tableur, il faut choisir le point-virgule comme séparateur de colonnes et le point comme séparateur décimal. Le journal du serveur et l'inventaire restent, eux, de simples fichiers texte.

Le bilan donne le nombre de mesures, leur moyenne, leur minimum et leur maximum. Pour la lumière, il indique la part des mesures valant 1 : cela ne correspond pas à une durée réelle d'allumage. Si une sauvegarde échoue, le programme signale l'erreur et n'ajoute pas la mesure concernée aux statistiques.

5. Alertes sur seuils

Configuration et déclenchement

Les alertes donnent au serveur un rôle d'analyse en plus de la collecte. Pour chaque type de capteur, il est possible de fixer un seuil bas, un seuil haut ou les deux. Les règles sont choisies dans `main` ; créer un serveur ne suffit donc pas à les activer. Les valeurs ci-dessous servent à la démonstration et ne représentent pas des normes réglementaires.

Grandeur	Déclenchement strict	Unité
Température	Valeur > 25	degC
Humidité	Valeur < 45	%
Son	Valeur > 70	dB

```
server.setAlertThresholds("Temperature", std::nullopt, 25.0, "degC");
server.setAlertThresholds("Humidity", 45.0, std::nullopt, "%");
server.setAlertThresholds("Sound", std::nullopt, 70.0, "dB");
```

Dans ces appels, `std::nullopt` signifie qu'aucun seuil n'est fixé de ce côté. Le programme refuse des bornes incohérentes, non finies ou toutes deux absentes. Redéfinir les seuils d'un type remplace sa règle précédente. L'unité est vérifiée avant l'enregistrement ; ces alertes numériques ne s'appliquent pas à la lumière booléenne.

Affichage et sauvegarde

Lorsqu'une mesure dépasse un seuil, le serveur affiche un message et ajoute une ligne à `alerts.csv`. Ce fichier apparaît à la première alerte dans le dossier de la simulation. Une valeur exactement égale au seuil est acceptée. Si le dépassement dure, chaque nouvelle mesure produit une nouvelle alerte : les répétitions ne sont pas encore filtrées.

```
Tick;SensorID;Type;Valeur;Unite;Direction;Seuil
3;2;Temperature;27;degC;HAUT;25
```

Cette ligne d'exemple permet de retrouver le capteur, le tick, la valeur et le seuil dépassé. Le bilan final affiche aussi le nombre d'alertes. Une mesure qui déclenche une alerte reste dans les statistiques : elle fait partie des données de la simulation.

Que se passe-t-il si l'écriture échoue ?

Si l'alerte ne peut pas être sauvegardée, le programme signale l'erreur sans augmenter son compteur ni mettre à jour les statistiques. La mesure peut cependant déjà figurer dans son propre CSV, car les deux fichiers sont écrits séparément. Lors d'une copie de serveur, les seuils et le compteur sont repris, mais les deux objets continuent d'écrire dans le même dossier.

6. Validation et résultats

Le programme a été compilé et testé sous Linux, via WSL, avec GNU C++ 15.2.0. La compilation en mode Debug et les tests lancés avec CTest ont réussi. Les options Wall, Wextra et Wpedantic sont activées pour repérer les avertissements dans la bibliothèque du simulateur.

Vérification automatisée	Résultat attendu
Température construite par défaut	Période de deux ticks
Copie, affectation, auto-affectation	Identité nouvelle ou conservée selon l'opération
Identifiants et paramètres invalides	Doublons, bornes, périodes et ticks refusés
Zéro tick et inscription en double	Aucune mesure ; une seule activation
Copie d'ordonnanceur	Références aux mêmes capteurs
Deux températures fixes 20 et 24	5 mesures, somme 108, min 20, max 24
Son, humidité, lumière sur six ticks	Respectivement 6, 2 et 6 mesures
Tirages analogiques et entiers	Valeurs dans les plages configurées
Évolution progressive	Écart maximal de 0,5 degC, 2 points, 5 dB
Alertes et seuils hauts/bas	Égalité, répétitions, CSV et copie vérifiés
Deux nouvelles sessions	Dossiers de journaux différents
Journaux et redirection de flux	En-tête CSV, tick et inventaire corrects
Destination indisponible	Erreur propagée, statistiques inchangées

Pour vérifier les calculs, les tests utilisent notamment des capteurs dont la valeur est fixe : les résultats attendus sont alors connus à l'avance. D'autres vérifications répètent les tirages pour contrôler les plages et les écarts entre deux mesures. Les valeurs aléatoires exactes ne sont pas comparées d'une exécution à l'autre. Toutes ces vérifications sont regroupées dans un test CTest.

Résultat de la démonstration

Sur les six ticks de la démonstration, le programme produit bien cinq mesures de température, six de son, deux d'humidité et six de lumière. Les trois types d'alerte apparaissent. Les fichiers permettent de retrouver l'origine et le tick de chaque mesure, puis le bilan rassemble les statistiques et le nombre d'alertes.

Ce qui reste à vérifier

Ces résultats concernent Linux/WSL. Les commandes Windows sont indiquées, mais la compilation native sous Windows reste à vérifier. Les tests couvrent les principaux cas prévus ; ils ne garantissent pas tous les comportements possibles en cas de panne de stockage ou de mauvaise utilisation des pointeurs.

7. Compilation, limites et bilan

Reproduire la compilation et les tests

Pour compiler le projet, il faut CMake 3.20 ou supérieur et un compilateur C++17 prenant en charge `std::filesystem`, par exemple GCC 9 ou supérieur, un MinGW récent ou Visual Studio 2022. CMake adapte les options d'avertissement à GNU ou MSVC. Les commandes sont les suivantes :

```
cmake -S . -B build-local -DCMAKE_BUILD_TYPE=Debug
cmake --build build-local --config Debug
ctest --test-dir build-local -C Debug --output-on-failure
```

L'exécutable se lance avec `./build-local/SimulateurIOT` sous Linux/WSL. Sous Windows, il se trouve dans `build-local/SimulateurIOT.exe` avec MinGW, ou dans `build-local/Debug/SimulateurIOT.exe` avec Visual Studio. La configuration VS Code fournie permet aussi de le déboguer avec GDB depuis une fenêtre WSL.

Limites et pistes d'évolution

- Les capteurs sont traités les uns après les autres. Un fonctionnement avec plusieurs threads demanderait de protéger les identifiants et les accès aux fichiers.
- Chaque capteur possède son compteur de temps. Une horloge commune faciliterait l'ajout de capteurs pendant la simulation.
- Les statistiques regroupent les mesures par type. Un bilan par capteur ou par pièce permettrait de mieux comparer les bureaux, en gardant des unités compatibles.
- Les variations progressives restent un modèle simple : elles ne reproduisent pas la physique d'une pièce et ne dépendent pas d'une durée réelle. Les alertes persistantes pourraient aussi être regroupées pour éviter les répétitions.
- Une erreur d'écriture arrête la démonstration. Le programme ne prévoit pas encore de reprise après une panne ni d'annulation des lignes déjà écrites.
- Les pointeurs supposent que les objets restent vivants pendant leur utilisation. Une gestion plus dynamique des capteurs demanderait de revoir leur propriété.

Bilan

La démonstration permet de suivre les cinq capteurs sur six ticks et de retrouver leurs mesures dans les fichiers CSV. Les deux capteurs de température ne mesurent pas au même rythme, mais leurs données arrivent au même serveur. Le scénario chaud, sec et bruyant rend les trois types d'alerte visibles dès cette courte exécution.

Les valeurs évoluent par petites variations, ce qui évite les changements brusques entre deux mesures. Le bilan final et les fichiers gardent une trace de ce qui s'est passé. Pour prolonger le projet, je commencerais par regrouper les alertes répétées : lorsqu'un capteur reste au-dessus du seuil, une nouvelle ligne apparaît à chaque mesure, ce qui peut vite remplir la console sur une simulation plus longue.